Grupa: _____ Numele şi prenumele (cu MAJUSCULE): _____

Arhitectura Sistemelor de Calcul: Parţial - 10% din nota finală

Toate foile suplimentare trebuie semnate!

Exerciţiul 1 **1.5p** *Scrieţi dimensiunea alocată în RAM, atât în `biţi`, cât şi în `Bytes`, pentru următoarele variabile declarate în secţiunea `.data`:*

Declarare în `.data`	***dimensiune în biţi***	***dimensiune în Bytes***
`str1: .asciz "Sir de\n"`		
`str2: .ascii "caractere\n"`		
`myWord: .long 242 8 35 17`		
`myLong: .long 0x8fde2025`		
`myBytes: .byte 's', 'i', 'r', '\0'`		

Scrieţi un apel de sistem `WRITE` (codul apelului este 4) pentru şirul de caractere `str1`, astfel încât să afişaţi, la `standard output`, întreg conţinutul şirului de caractere şi nimic altceva în plus.

Exerciţiul 2 **1p** *Completaţi următorul tabel, cu valorile corecte în bazele 2 şi 16.*

Baza 2	***Baza 16***
	04 00 00 02
1111 0101 1010 1100 0101 0011 1010 1010	

Exerciţiul 3 **2p** *Fie următorul program, scris în `assembly` (procesor Intel x86, sintaxa `AT&T`):*

```
.data
    x: .long 0x4000002
    z: .long 0x2000
    t: .long 0b100 0000 0000 0000
.text
.global main
main:
    movl x, %eax
    mull z
et1:
    divl t
    decl %eax
et_exit:
    movl $1, %eax
    xorl %ebx, %ebx
    int $0x80
```

Executăm, în `debugger`, următoarele comenzi:

```
b main; b et_exit; run; stepi; stepi; i r eax; i r edx; c; i r eax; i r edx;
```

Valorile pe care le vom obţine sunt, în ordine:

EAX (1)	***EDX (1)***	***EAX (2)***	***EDX (2)***

Important! *Completaţi valorile exclusiv în baza 16.*

Exerciţiul 4 **1.5p** *Fie următoarele instrucţiuni în `main`:*

```
movl $0x8000000, %eax
movl $0x200000, %ebx
mul %ebx
divl p
```

Descrieţi ce obţinem dacă $p = 2^{14}$. Dar dacă $p = 2^{20}$?

Exerciţiul 5 **1p** *Fie următorul program, scris în* `assembly` *(procesor* `Intel x86`*, sintaxa* `AT&T`*):*

```
.data
 x: .long 5
 y: .long 20
.text
.global main
main:
    movl x, %ebx
    movl y, %eax
    movl $15, %edx
    mul %ebx
    add %edx, %eax
    movl $2, %esi
    divl %esi
    movl %edx, %ecx
    addl $20, %ecx
et_loop:
    inc %ebx
    inc %edx
    dec %eax
    loop et_loop
    add %edx, %eax
et_exit:
    movl $1, %eax
    xorl %ebx, %ebx
    int $0x80
```

*Completaţi valorile regiştrilor (**in baza 10**) în momentul în care executarea programului ajunge în dreptul etichetei* `et_exit` *(după ce am rulat în* `debugger` *comenzile* `b et_exit; run; i r`*):*

EAX	***EBX***	***EDX***

Exerciţiul 6 **1p** *Fie următorul program, dezvoltat în limbajul de asamblare al procesorului* `Intel x86`*, sintaxa* `AT&T`*:*

```
.data
.text
.global main
main:
    movl $0xd8eb6d7f, %ecx
    sub %eax, %eax
    sub %ebx, %ebx
    movl $0, %edx
    movw %cx, %dx
    movb %dh, %al
    movb %dl, %bh
    addw %ax, %bx
    cmp %ebx, %edx
    jg et
et_loop:
    cmp $0, %eax
    je et_exit
    dec %ebx
    dec %eax
    jmp et_loop
et:
    cmp $0, %eax
    je et_exit
    inc %ebx
    dec %eax
    jmp et_loop
et_exit:
    movl $1, %eax
    xorl %ebx, %ebx
    int $0x80
```

Valoarile stocate în dreptul etichetei `et_cont` *(în baza 16) sunt:*

EAX	***EBX***	***ECX***	***EDX***

Exerciţiul 7 **1p** *Scrieţi două moduri **distincte** prin care puteţi accesa elementul al 3-lea dintr-un array de* `.word`*-uri. (presupuneţi că există elementul al 3-lea - cel cu indexul 2, fiind considerată indexarea de la 0) Array-ul există în secţiunea data şi se numeşte* `w`*.*

Exerciţiul 8 **1p** *Scrieţi o secvenţă de cod în* `main` *care să calculeze produsul resturilor împărţirii la 5 a pătratelor elementelor dintr-un array ($\prod_{i=0}^{n-1}((v[i])^2 \bmod 5)$). Consideraţi că lungimea array-ului este* `n`*, iar numele lui este* `v`*. Aceste informaţii sunt deja în secţiunea* `.data`*. Soluţia voastră trebuie să înceapă de la eticheta* `main`*.*